

Assignment 2

Information Theory for Data Science

328158, Maleki, Amir
328728, Masteri Farahani, Farzaneh
323337, Akhane, Mohammad Amin
327167, Noshiravanbaboli, Yasaman

December 12, 2023

Data Science and Engineering

Politecnico di Torino

2023-2024

1 Basic Notions

1.1 Calculate the entropy of the following distributions:

1.1.1 :

$$p(n) \propto \exp(-\lambda n), \quad n = 0, 1, 2, \dots$$

$p(n)$ is proportional to the exponential distribution above. In order to have a valid probability distribution, we should normalize it. It means that the sum of probabilities of possible values of n should be equal to 1. We can find the normalization constant C as below:

$$\sum_{n=0}^{\infty} p(n) = \sum_{n=0}^{\infty} C \exp(-\lambda n)$$

The sum converges if $\lambda > 0$ as shown in figure 1, and it diverges when $\lambda < 0$. So we suppose λ a positive number in order to have a valid probability function.

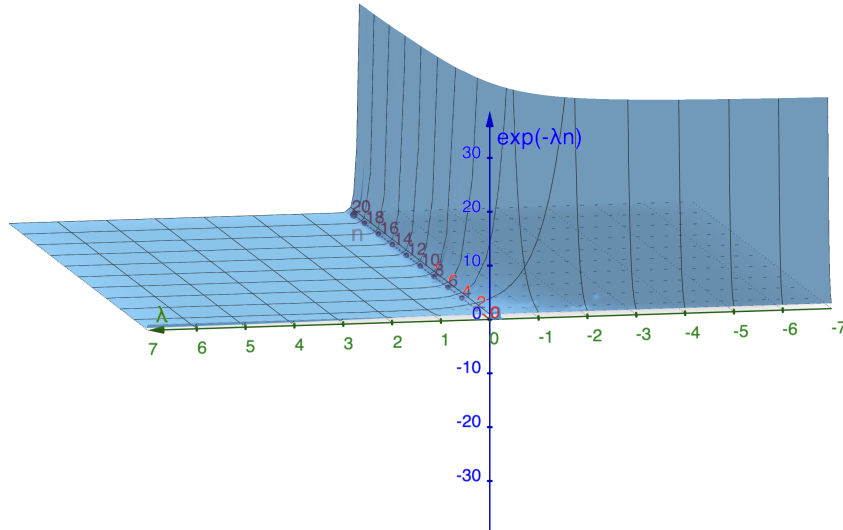


Figure 1: $\exp(-\lambda n)$

The normalization constant can be calculated as below:

$$C = \frac{1}{\sum_{n=0}^{\infty} \exp(-\lambda n)} = 1 - \exp(-\lambda)$$

Proof.

$$\begin{aligned} A &= \sum_{n=0}^{\infty} \exp(-\lambda n) \\ &= \frac{1}{e^0} + \frac{1}{e^\lambda} + \frac{1}{e^{2\lambda}} + \dots \\ \frac{A}{e^\lambda} &= \frac{1}{e^\lambda} + \frac{1}{e^{2\lambda}} + \frac{1}{e^{3\lambda}} + \dots \\ &= A - 1 \\ A &= \frac{e^\lambda}{e^\lambda - 1} \end{aligned}$$

□

So, the normalized probability mass function is:

$$p(n) = \exp(-\lambda n)(1 - \exp(-\lambda)), \quad \lambda > 0$$

Now we calculate the entropy of the normalized probability mass function:

$$\begin{aligned}
H(n) &= - \sum_{n=0}^{\infty} p(n) \log_2(p(n)) \\
&= - \sum_{n=0}^{\infty} \exp(-\lambda n)(1 - \exp(-\lambda)) \log_2(\exp(-\lambda n)(1 - \exp(-\lambda))) \\
&= -(1 - \exp(-\lambda)) \sum_{n=0}^{\infty} \exp(-\lambda n) (-\lambda n \log_2(e) + \log_2(1 - \exp(-\lambda))) \\
&= -(1 - \exp(-\lambda)) \left[\sum_{n=0}^{\infty} \exp(-\lambda n) (-\lambda n \log_2(e)) + \log_2(1 - \exp(-\lambda)) \sum_{n=0}^{\infty} \exp(-\lambda n) \right] \\
&= -(1 - \exp(-\lambda)) \left[-\lambda \log_2(e) \sum_{n=0}^{\infty} n \exp(-\lambda n) + \log_2(1 - \exp(-\lambda)) \sum_{n=0}^{\infty} \exp(-\lambda n) \right] \\
&= -(1 - \exp(-\lambda)) \left[-\lambda \log_2(e) \frac{\exp(1 - \lambda)}{(1 - \exp(-\lambda))(e - 1)} + \log_2(1 - \exp(-\lambda)) \frac{\exp(-\lambda)}{1 - \exp(-\lambda)} \right] \\
&= \lambda \log_2(e) \exp(1 - \lambda) - \exp(-\lambda) \log_2(1 - \exp(-\lambda))
\end{aligned}$$

1.1.2 :

$$p(n) \propto \exp(-n^2), \quad n = 0, 1, 2, \dots$$

For this probability function the analytical solution is impossible. Therefore, below we can see the numerical solution in Matlab:

```

1 % Define the probability function
2 n = 0:10;
3 pn = zeros(1,11);
4
5 for i = 1:length(pn)
6     pn(i) = exp(-n(i)^2);
7 end
8
9 % Normalize the probability function
10 pn_normalized = pn ./ sum(pn);
11
12 % Calculate the entropy
13 entropy = -sum(pn_normalized .* log2(pn_normalized));
14
15 disp('Entropy:');
16 disp(entropy);

```

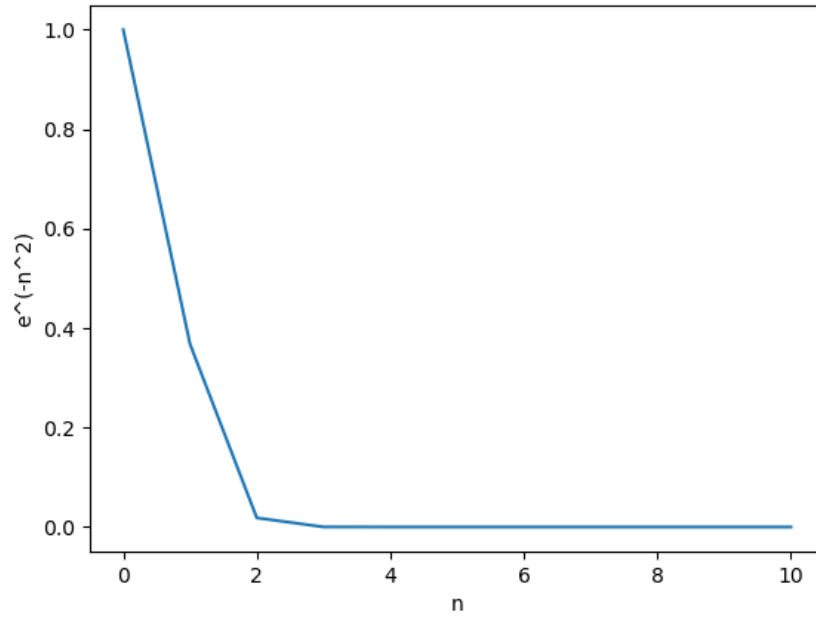


Figure 2: $\exp(-n^2)$

As $p(n)$ for $n > 10$ has a negligible amount, we compute it numerically in the range of 0 to 10. The entropy equals to 0.9315.

1.1.3 :

$$p(n) \propto n^{-4}, \quad n = 1, 2, 3, \dots$$

```

1 % Define the probability function
2 n = 1:10;
3 pn = zeros(1,10);
4
5 for i = 1:length(pn)
6     pn(i) = n(i)^-4;
7 end
8
9 % Normalize the probability function
10 pn_normalized = pn ./ sum(pn);
11
12 % Calculate the entropy
13 entropy = -sum(pn_normalized .* log2(pn_normalized));
14
15 disp('Entropy:');
16 disp(entropy);

```

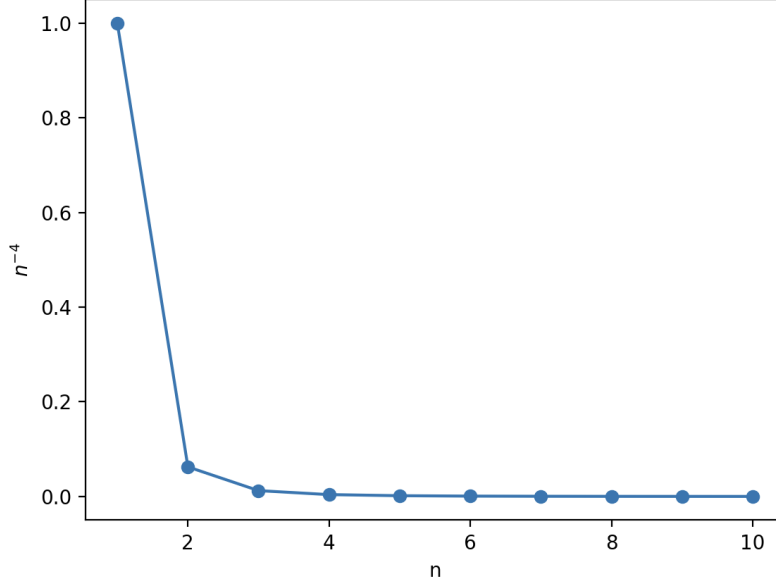


Figure 3: n^{-4}

As $p(n)$ for $n > 10$ has a negligible amount, we compute it numerically in the range of 0 to 10. The entropy equals to 0.4772.

1.1.4 :

$$p(n) \propto \alpha^n, \quad n = 1, \dots, N$$

$p(n)$ is proportional to the exponential distribution above. In order to have a valid probability distribution, we should normalize it. It means that the sum of probabilities of possible values of n should be equal to 1. We can find the normalization constant C as below:

$$\sum_{n=1}^N p(n) = \sum_{n=1}^N C \alpha^n$$

The normalization constant can be calculated as below:

$$C = \frac{1}{\sum_{n=1}^N \alpha^n}$$

$$\sum_{n=1}^N \alpha^n = \alpha^1 + \alpha^2 + \dots + \alpha^N = \frac{\alpha(\alpha^N - 1)}{\alpha - 1}$$

So, the normalized probability mass function is:

$$p(n) = \frac{(\alpha - 1)\alpha^{n-1}}{\alpha^N - 1}, \quad 0 < \alpha < 1$$

$$\begin{aligned}
H(n) &= - \sum_{n=1}^N p(n) \log_2(p(n)) \\
&= - \sum_{n=1}^N C\alpha^n \log_2(C\alpha^n) \\
&= -C[\log_2(C \sum_{n=1}^N N\alpha^n) + \log_2(\alpha \sum_{n=1}^N N\alpha^n)] \\
&= \frac{\alpha-1}{\alpha(\alpha^N-1)} [\log_2(\frac{\alpha-1}{\alpha(\alpha^N-1)} \frac{\alpha(1-\alpha^N)}{1-\alpha}) + \log_2(\alpha) \frac{1}{\alpha-1} (N\alpha^{N+1} - \frac{\alpha(1-\alpha^N)}{1-\alpha})] \\
&= -\log_2(\frac{\alpha-1}{\alpha(\alpha^N-1)}) + \log_2(\alpha) (\frac{N\alpha}{\alpha^N-1} + \frac{1}{1-\alpha})
\end{aligned}$$

1.1.5 :

$$p(n) \propto (1+n^2)^{-k}, \quad n = 0, 1, 2, \dots \quad \text{for } k = 1, 2$$

```

1 %% For K = 1
2 % Define the probability function
3 n = 0:10000;
4 pn = zeros(1,10001);
5
6 for i = 1:length(pn)
7     pn(i) = (1+n(i)^2)^-1;
8 end
9
10 % Normalize the probability function
11 pn_normalized = pn ./ sum(pn);
12
13 % Calculate the entropy
14 entropy = -sum(pn_normalized .* log2(pn_normalized));
15
16 disp('Entropy:');
17 disp(entropy);
18
19 %% For K = 2
20 % Define the probability function
21 n = 0:50;
22 pn = zeros(1,51);
23
24 for i = 1:length(pn)
25     pn(i) = (1+n(i)^2)^-2;
26 end
27
28 % Normalize the probability function
29 pn_normalized = pn ./ sum(pn);
30
31 % Calculate the entropy
32 entropy = -sum(pn_normalized .* log2(pn_normalized));
33
34 disp('Entropy:');
35 disp(entropy);

```

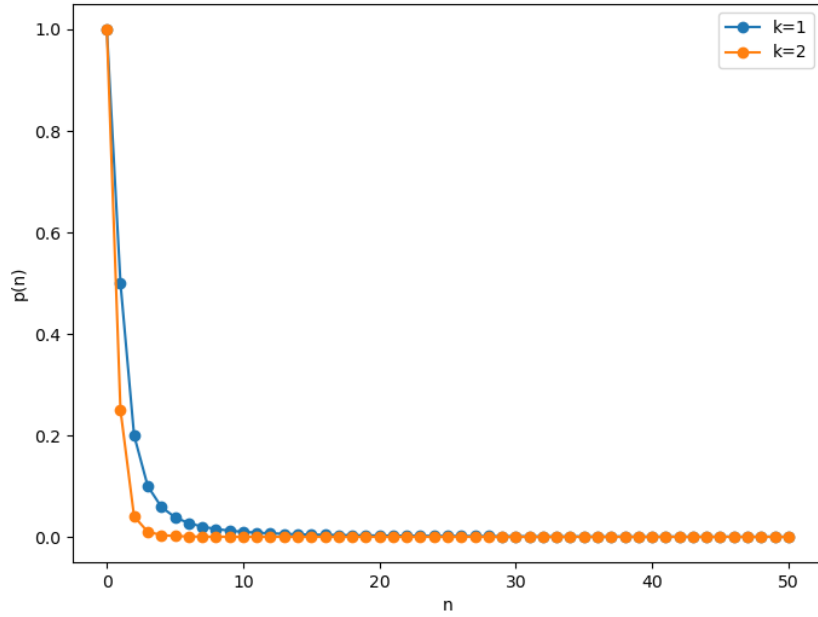


Figure 4: $(1 + n^2)^{-k}$

When $k=2$ $p(n)$ for $n > 50$ has a negligible amount, so we compute it numerically in the range of 0 to 50. The entropy equals to 1.0112.

But for $k=1$ we use n from 0 to 10000 to compute numerically. The entropy equals to 2.5583.

1.2 Calculate the differential entropy of the following pdfs:

1.2.1 :

$$f_X(x) = \frac{u(x-a) - u(x-b)}{b-a}$$

As we know $u(x)$ can be defined as:

$$u(x) = \begin{cases} 0 & \text{if } x < 0, \\ 1 & \text{if } x \geq 0. \end{cases}$$

In this problem we can have two situations, first is when $a > b$ and second $b > a$. In the first situation ($a > b$) we have:

$$\frac{u(x-a) - u(x-b)}{b-a} = \begin{cases} \frac{1}{b-a} & \text{if } b < x < a, \\ 0 & \text{otherwise} \end{cases}$$

,And In the second situation ($b > a$) we have:

$$\frac{u(x-a) - u(x-b)}{b-a} = \begin{cases} \frac{-1}{b-a} & \text{if } a < x < b, \\ 0 & \text{otherwise} \end{cases}$$

So generally we can write:

$$f_X(x) = \begin{cases} \frac{1}{|b-a|} & \text{if } x \in [a, b], \\ 0 & \text{otherwise} \end{cases}$$

So we can calculate the differential entropy as below:

$$\begin{aligned}
h(X) &= - \int_{-\infty}^{\infty} f_X(x) \log_2 f_X(x) dx \\
&= - \int_a^b f_X(x) \log_2 f_X(x) dx \\
&= -(a-b) \frac{1}{|a-b|} \log_2 \frac{1}{|a-b|} \\
&= \frac{(a-b)}{|a-b|} \log_2 |a-b|
\end{aligned}$$

1.2.2 :

$$f_X(x) = \lambda e^{-\lambda x} u(x)$$

As we know $u(x)$ can be defined as:

$$u(x) = \begin{cases} 0 & \text{if } x < 0, \\ 1 & \text{if } x \geq 0. \end{cases}$$

In this problem we can have two situations, first if $x \geq 0$ and second if $x < 0$, in general we have:

$$f_X(x) = \begin{cases} 0 & x < 0 \\ \lambda e^{-\lambda x} & x \geq 0 \end{cases}$$

So we can calculate the differential entropy as below:

$$\begin{aligned}
h(X) &= - \int_{-\infty}^{\infty} f_X(x) \log_2 f_X(x) dx \\
&= - \int_0^{\infty} \lambda e^{-\lambda x} \log_2 \lambda e^{-\lambda x} dx
\end{aligned}$$

We can simplify that:

$$\log_2 \lambda e^{-\lambda x} = \log_2 \lambda - \lambda x \text{ for } x \geq 0$$

So we can calculate as below:

$$\begin{aligned}
h(X) &= - \int_0^{\infty} \lambda e^{-\lambda x} (\log_2 \lambda - \lambda x) dx \\
&= - \lim_{t \rightarrow \infty} \int_0^t \lambda e^{-\lambda x} (\log_2 \lambda - \lambda x) dx \\
&= - \lim_{t \rightarrow \infty} \lambda \int_0^t e^{-\lambda x} (\log_2 \lambda - \lambda x) dx
\end{aligned}$$

Assuming $\int u dv = uv - \int v du$ and $u = \log_2(\lambda)$, $dv = e^{-\lambda x}$ we can write:

$$\begin{aligned}
h(X) &= - \int_0^\infty \lambda e^{-\lambda x} (\log_2 \lambda - \lambda x) dx \\
&= - \lim_{t \rightarrow \infty} \lambda \left[(\log_2 \lambda - \lambda x) \left(-\frac{e^{-\lambda x}}{\lambda} \right) \Big|_0^t - \int_0^t -\frac{e^{-\lambda x}}{\lambda} (-\lambda) dx \right] \\
&= - \lim_{t \rightarrow \infty} \lambda \left[(\log_2 \lambda - \lambda x) \left(-\frac{e^{-\lambda x}}{\lambda} \right) \Big|_0^t - \int_0^t e^{-\lambda x} dx \right]
\end{aligned}$$

Assuming $u = -\lambda x$ then $du = -\lambda dx$, so $dx = -\frac{1}{\lambda} du$ so we can write:

$$\begin{aligned}
h(X) &= - \int_0^\infty \lambda e^{-\lambda x} (\log_2 \lambda - \lambda x) dx \\
&= - \lim_{t \rightarrow \infty} \lambda \left[(\log_2 \lambda - \lambda x) \left(-\frac{e^{-\lambda x}}{\lambda} \right) \Big|_0^t - \int_0^{-\lambda t} -\frac{e^u}{\lambda} du \right] \\
&= - \lim_{t \rightarrow \infty} \lambda \left[(\log_2 \lambda - \lambda x) \left(-\frac{e^{-\lambda x}}{\lambda} \right) \Big|_0^t + \frac{1}{\lambda} \int_0^{-\lambda t} e^u du \right] \\
&= - \lim_{t \rightarrow \infty} \lambda \left[(\log_2 \lambda - \lambda x) \left(-\frac{e^{-\lambda x}}{\lambda} \right) \Big|_0^t + \frac{1}{\lambda} e^u \Big|_0^{-\lambda t} \right] \\
&= - \lim_{t \rightarrow \infty} (\log_2 \lambda - \lambda t) (-e^{-\lambda t}) + \log_2 \lambda + e^{-\lambda t} - 1
\end{aligned}$$

And The final result would be:

$$\begin{aligned}
h(X) &= - \int_0^\infty \lambda e^{-\lambda x} (\log_2 \lambda - \lambda x) dx \\
&= - (\log_2 \lambda - 1) \\
&= 1 - \log_2 \lambda
\end{aligned}$$

1.2.3 :

$$f_X(x) = \frac{1}{\Gamma(n)} x^{n-1} e^{-x} u(x)$$

As we know $u(x)$ can be defined as:

$$u(x) = \begin{cases} 0 & \text{if } x < 0, \\ 1 & \text{if } x \geq 0. \end{cases}$$

We can rewrite the pdf as below with respect to the $u(x)$:

$$f_X(x) = \begin{cases} 0 & \text{if } x < 0, \\ \frac{1}{\Gamma(n)} x^{n-1} e^{-x} & \text{if } x \geq 0. \end{cases}$$

Also, Gamma function $\Gamma(n)$ define as $\Gamma(n) = (n-1)!$ for every positive integer n .

The differential entropy of the pdf is as below:

$$h(X) = - \int_{-\infty}^{\infty} f_X(x) \log_2 f_X(x) dx$$

Then we have:

$$h(X) = - \int_0^{\infty} \frac{1}{(n-1)!} x^{n-1} e^{-x} \log_2 \frac{1}{(n-1)!} x^{n-1} e^{-x} dx$$

To simplify the Logarithm, we can do:

$$\begin{aligned} \log_2 \frac{1}{(n-1)!} x^{n-1} e^{-x} &= \log_2 \frac{1}{(n-1)!} + (n-1) \log_2 x - x \log_2 e \\ &= \log_2 \frac{1}{(n-1)!} + (n-1) \log_2 x - x \end{aligned}$$

Now we have new $h(x)$ equation:

$$h(X) = - \int_0^{\infty} \frac{1}{(n-1)!} x^{n-1} e^{-x} (\log_2 \frac{1}{(n-1)!} + (n-1) \log_2 x - x) dx$$

At the next step, we split the integral into the three integrals and calculate each of them separately:

1. $\int_0^{\infty} \frac{1}{(n-1)!} x^{n-1} e^{-x} \log_2 \frac{1}{(n-1)!} dx$
2. $(n-1) \int_0^{\infty} x^{n-1} e^{-x} \log_2 x dx$
3. $-\int_0^{\infty} \frac{1}{(n-1)!} x^n e^{-x} dx$

For the **First** one we have:

$$\int_0^{\infty} \frac{1}{(n-1)!} x^{n-1} e^{-x} \log_2 \frac{1}{(n-1)!} dx = \frac{1}{(n-1)!} \log_2 \frac{1}{(n-1)!} \int_0^{\infty} x^{n-1} e^{-x} dx$$

The integral part is definition of Gamma Function $\Gamma(n)$, so we have:

$$\frac{1}{(n-1)!} \log_2 \frac{1}{(n-1)!} \Gamma(n)$$

For the **Second** one we have:

$$(n-1) \int_0^\infty x^{n-1} e^{-x} \log_2 x dx$$

Assume $u = \log_2 x$ and $dv = x^{n-1} e^{-x} dx$, applying $\int u dv = uv - \int v du$ to the corresponding equations $du = \frac{1}{x \ln 2}$ and $v = -e^{-x} \frac{x^n}{n}$:

$$\begin{aligned} &= (n-1) \left[-\frac{e^{-x} x^n}{n} \log_2 x \Big|_0^\infty + \frac{1}{n} \int_0^\infty \frac{e^{-x} x^n}{x \ln 2} dx \right] \\ &= (n-1) \int_0^\infty x^{n-1} e^{-x} \log_2 x dx \\ &= \frac{n-1}{n} \int_0^\infty x^{n-1} e^{-x} dx \end{aligned}$$

The integral part is actually the definition of Gamma function $\Gamma(n)$, so the result is:

$$\frac{n-1}{n} \Gamma(n)$$

For the **Third** one:

$$- \int_0^\infty \frac{1}{(n-1)!} x^n e^{-x} x dx$$

Assume that $u = x$ and $dv = \frac{1}{(n-1)!} x^n e^{-x} dx$, then $du = dx$ and $v = -e^{-x}$, applying the integration:

$$\begin{aligned} &= - \left[\frac{e^{-x} x^{n+1}}{n(n-1)!} \Big|_0^\infty - \frac{1}{n(n-1)!} \int_0^\infty e^{-x} x^{n+1} dx \right] \\ &= 0 - \frac{1}{n(n-1)!} \int_0^\infty e^{-x} x^{n+1} dx \end{aligned}$$

The integral can be expressed in terms of the Gamma function:

$$- \frac{1}{n(n-1)!} \Gamma(n+2)$$

Now, combining results of three parts we get:

$$\begin{aligned} h(X) &= - \int_0^\infty \frac{1}{(n-1)!} x^{n-1} e^{-x} \left(\log_2 \frac{1}{(n-1)!} + (n-1) \log_2 x - x \right) dx \\ &= - \left[\frac{1}{(n-1)!} \log_2 \frac{1}{(n-1)!} \Gamma(n) + \frac{n-1}{n} \Gamma(n) - \frac{1}{n(n-1)!} \Gamma(n+2) \right] \\ &= - \frac{1}{(n-1)!} \log_2 \frac{1}{(n-1)!} \Gamma(n) - \frac{n-1}{n} \Gamma(n) + \frac{1}{n(n-1)!} \Gamma(n+2) \end{aligned}$$

2 Markov Source

In the first step we sum up the outputs of the function for each x_n over all values of $x_{n-3:n-1}$ to calculate the normalization constant. Then we can calculate the probability function by using the normalization constant.

Then we should solve the equation $p_\infty = p_\infty.P$ by finding the eigenvectors and eigenvalues, and using the eigenvector corresponding to the eigenvalue 1. Finally for calculating entropy rate($\bar{H}$) we have the definition below:

$$\bar{H} = - \sum_{\sigma \in X^L} P_\infty(\sigma) \sum_{x \in X} p(x|\sigma) \log_2 p(x|\sigma)$$

```

1 % Initialize the transition probability matrix
2 P = zeros(125,125);
3
4 %% normalizing and calculating the Probability matrix
5 iter = 1;
6 for xn_3 = 1:5
7     for xn_2 = 1:5
8         for xn_1 = 1:5
9             row_sum = 0;
10            for xn = 1:5
11                row_sum = row_sum + (max([xn_3, xn_2, xn_1]))^(xn/5);
12            end
13            for xn = 1:5
14                P(state_to_num(xn_3, xn_2, xn_1), state_to_num(xn_2,
15                    xn_1, xn)) = (max([xn_3, xn_2, xn_1]))^(xn/5) / row_sum;
16            end
17        end
18    end
19
20 %% Calculate asymptotic state distribution
21
22 % First we Calculate left eigenvectors and eigenvalues
23 [eigenvectors, eigenvalues] = eig(P. ');
24
25 % then we find the index of the eigenvalue 1
26 epsilon = 0.0001;
27 eigenvalue1_index = find(abs(diag(eigenvalues) - 1) < epsilon);
28
29 % then we find the corresponding eigenvector to eigenvalue 1
30 p_infinity = eigenvectors(:, eigenvalue1_index)';
31
32 % finally we normalize the p_inf, so that the sum of all values=1
33 p_infinity = p_infinity / sum(p_infinity);
34
35 disp('Asymptotic state distribution =');
36 disp(p_infinity);
37
38 %% calculating entropy rate
39 H_bar = 0;
40 for i = 1:length(p_infinity)
41     sum_conditional_dists = 0;
42     [x3, x2, x1] = num_to_state(i);
43     for j = 1:5

```

```

44         number_of_state = state_to_num(x2, x1, j);
45         conditional_dist = P(i, number_of_state) * log2(P(i,
number_of_state));
46         sum_conditional_dists = sum_conditional_dists +
conditional_dist;
47     end
48     H_bar = H_bar - p_infinity(i)*sum_conditional_dists;
49 end
50
51 disp("Entropy Rate: H_bar = ")
52 disp(H_bar);
53
54 %% functions
55 % this function gets a state number and convert it to the corresponding
56 % state
57 function [x3, x2, x1] = num_to_state(num)
58     x3 = floor((num-1)/25) + 1;
59     num = num - (x3-1)*25;
60     x2 = floor((num-1)/5) + 1;
61     num = num - (x2-1)*5;
62     x1 = num;
63 end
64
65 % this function gets a state and convert it to the corresponding state
66 % number
67 function num = state_to_num(x3, x2, x1)
68     num = 0;
69     num = num + (x3-1)*25;
70     num = num + (x2-1)*5;
71     num = num + x1;
72 end

```

3 Huffman Code

For the stationary source we can compute probabilities with

$$p(n) \propto \frac{1}{1+n^3} \text{ which } n = 0, 1, 2, 3, \dots, N-1.$$

At the first step, we have to normalize these probabilities to make sure that the sum of them is equal to 1.

So we use this equation to normalize the probabilities:

$$P(n) = \frac{p(n)}{\sum_{i=0}^{N-1} p(i)}$$

We do it with the **calculate_probabilities** function of the code to calculate the normalized probabilities and as the return, we get the normalized probability.

After obtaining the normalized probabilities, we have to determine the Entropy rate using equation below:

$$H(X) = - \sum_{i=0}^{N-1} P(i) * \log_2 P(i)$$

We do this by defining **calculate_entropy** function which gets probabilities as input and calculate and return the entropy of corresponding probabilities as output.

Then, we have to calculate the average number of bits, for this purpose we implement the **huffman_code_lengths** function which gets the normalized probabilities as input and return the length of huffman code for each of the probabilities.

After these steps, we have to calculate the average number of bits per symbol, we perform a element-wise multiplication between the vectors of normilized probabilities and the average number of bits and finally we calculate sum of all elements in the multiplication vector.

```
1 %% Calculate For N=10
2
3 % Calculating probabilities
4 probabilities_1 = calculate_probabilities(10);
5
6 % Calculating entropy
7 entropy_1 = calculate_entropy(probabilities_1);
8
9 % Calculating Huffman code lengths
10 huffman_code_lengths_1 = huffman_code_lengths(probabilities_1);
11
12 % Calculating the average number of bits per symbol
13 average_bits_1 = sum(probabilities_1 * huffman_code_lengths_1);
14
15 fprintf('ENTROPY RATE VALUE FOR N=10 IS %f\n', entropy_1);
16 fprintf('THE AVERAGE NUMBER OF BITS PER SYMBOL FOR N=10 IS %f\n',
    average_bits_1);
17
18 %% Calculate For N=100
19
20 % Calculating probabilities
21 probabilities_2 = calculate_probabilities(100);
22
23 % Calculating entropy
24 entropy_2 = calculate_entropy(probabilities_2);
25
26 % Calculating Huffman code lengths
27 huffman_code_lengths_2 = huffman_code_lengths(probabilities_2);
28
29 % Calculating the average number of bits per symbol
30 average_bits_2 = sum(probabilities_2 * huffman_code_lengths_2);
31
32 fprintf('\nENTROPY RATE VALUE FOR N=100 IS %f\n', entropy_2);
33 fprintf('THE AVERAGE NUMBER OF BITS PER SYMBOL FOR N=100 IS %f\n',
    average_bits_2);
34
35 %% Calculate For N=1000
36
37 % Calculating probabilities
38 probabilities_3 = calculate_probabilities(1000);
39
40 % Calculating entropy
41 entropy_3 = calculate_entropy(probabilities_3);
42
43 % Calculating Huffman code lengths
```

```

44 huffman_code_lengths_3 = huffman_code_lengths(probabilities_3);
45
46 % Calculating the average number of bits per symbol
47 average_bits_3 = sum(probabilities_3 * huffman_code_lengths_3);
48
49 fprintf('\nENTROPY RATE VALUE FOR N=1000 IS %f\n', entropy_3);
50 fprintf('THE AVERAGE NUMBER OF BITS PER SYMBOL FOR N=1000 IS %f\n',
    average_bits_3);
51 %% Normalization
52
53 % For the stationary source, we can compute probabilities with  $p(n)$ 
     $1/(1+n^3)$ , where  $(n = 0, 1, \dots, N-1)$ .
54 % We need to normalize these probabilities to make sure that they sum
    up to 1.
55 % We can use this equation to normalize the probabilities in order to
    achieve our goal.
56
57 %% Defining a function which calculates the normalized probabilities
58 function normalized_probabilities = calculate_probabilities(N)
59     probabilities = zeros(1, N);
60
61     for n = 1:N
62         probabilities(n) = 1 / (1 + (n-1)^3);
63     end
64
65     normalization_constant = sum(probabilities);
66     normalized_probabilities = probabilities / normalization_constant;
67 end
68
69 %% Entropy Rate
70
71 % After normalizing the probabilities, we can determine the Entropy
    Rate With this equation.
72
73 function entropy = calculate_entropy(probabilities)
74     entropy = -sum(probabilities .* log2(probabilities));
75 end
76
77 %% Average number of bits
78
79 % In each iteration we choose the two minimum probabilities, and
    combine
80 % them into a node with the sum of their probabilities
81
82
83 function length_encoded_words = huffman_code_lengths(probs)
84
85     encoded_words = configureDictionary("string","string");
86     probabilities = configureDictionary("string","double");
87     index = [1:length(probs)];
88     i = 1;
89     for p = probs
90         probabilities(string(index(i))) = p;
91         i = i + 1;
92     end
93
94     while length(probabilities.keys)~=1
95         [min1_value, min1_prob] = findMin(probabilities);

```

```

96     probabilities = remove(probabilities, min1_prob);
97
98     for w = min1_prob.split(",")
99         if isKey(encoded_words, w)
100             encoded_words(w) = "0" + encoded_words(w);
101         else
102             encoded_words(w) = "0";
103         end
104     end
105
106     [min2_value, min2_prob] = findMin(probabilities);
107     probabilities = remove(probabilities, min2_prob);
108
109     for w = min2_prob.split(",")
110         if isKey(encoded_words, w)
111             encoded_words(w) = "1" + encoded_words(w);
112         else
113             encoded_words(w) = "1";
114         end
115     end
116
117     probabilities(min1_prob+","+min2_prob) = min1_value +
min2_value;
118     end
119     length_encoded_words = strlen(encoded_words.values);
120     length_encoded_words = flip(length_encoded_words);
121 end
122
123 function [value, word] = findMin(array)
124     values = array.values;
125     words = array.keys;
126
127     value = values(1);
128     word = words(1);
129     for i = 1:length(values)
130         if values(i) < value
131             value = values(i);
132             word = words(i);
133         end
134     end
135 end

```

We know that the entropy rate is bound by the range $[0, \log_2(N)]$, where N is the number of possible symbols in the process. In practice, if the entropy rate is close to 0, it suggests a highly ordered or deterministic process, while a value close to $\log_2(N)$ indicates high randomness or unpredictability.

In this exercise:

- When N is equal to 10, the entropy rate is 1.500516 which it indicates a high entropy rate and means that the symbols it produces are less predictable.
- When N is equal to 100, the entropy rate is 1.541135 which is a medium entropy rate and indicates a process with moderate randomness or unpredictability.
- When N is equal to 1000, the entropy rate is 1.541806 which is a low entropy rate and indicates this process has very little uncertainty.

The goal of prefix-free tree codes, specifically those generated by algorithms like Huffman coding, is to minimize the average number of bits required to represent symbols in a given data set.

In this exercise we can observe that:

- When the number of symbols is 10, the average number of bits is 1.597947.
- When the number of symbols is 100, the average number of bits is 1.634303.
- When the number of symbols is 1000, the average number of bits is 1.634940.

So the result is the average number of bits is independent on number of bits and in any condition the Huffman code can minimize it perfectly.

4 Text File Encoding

In the first step we use the function *preprocess* to upper case, remove accents, remove punctuations and the text into words. Then we built a dictionary of the words with their frequencies, and calculated the entropy rate with the function *calculate_entropy_rate* by using the frequency of the words as probabilities. entropy rate = 9.6547

Then for all values of $N \in 1000, 2000, 3000, \dots, 20000$ we do the process below. First we sort the words based on their frequencies and encode the first N frequent ones with the function *encode_huffman* and save the encoded values of huffman tree in *huffman.txt* with this format in each line *Word, Huffman Code*.

Then we start the encoding of all the given context. We iterate on each word and if it is encoded with huffman, we use the huffman code which we load from the *huffman.txt* file. If the word is not included in huffman tree, we encode it with max length encoding. For max length encoding we just encode each character into 6 bits because we have only 36 characters. We need to add some other bits to the first of the encodings.

For huffman encodings it will be in this format:

[flag_bit = 0, length of the encoding(8bits), encoding]

For max length encodings it will be in this format:

[flag_bit = 1, length of the encoding(8bits), encoding]

It is good to mention that we use 8 bits because the maximum length between given words was 27 characters and we use 6 bits for encoding each character. So we have $\text{ceil}(\log_2(27 * 6)) = 8$. Finally the encoded text will be saved in a binary format in the file *encoded_data.bin* using the function *save_encoded_data*. The decoding will be simple, as we start from the beginning of the encoded data, we find out the kind of encoding with flag bit and then the next 8 bits shows the length of following encoding.

**It is not necessary to add length bits for huffman encodings but as Professor Taricco said in the website, we added the length bits for both max length and huffman encodings.*

The comparison of memory usage compression percentage based on different values of N is shown in the plot below.

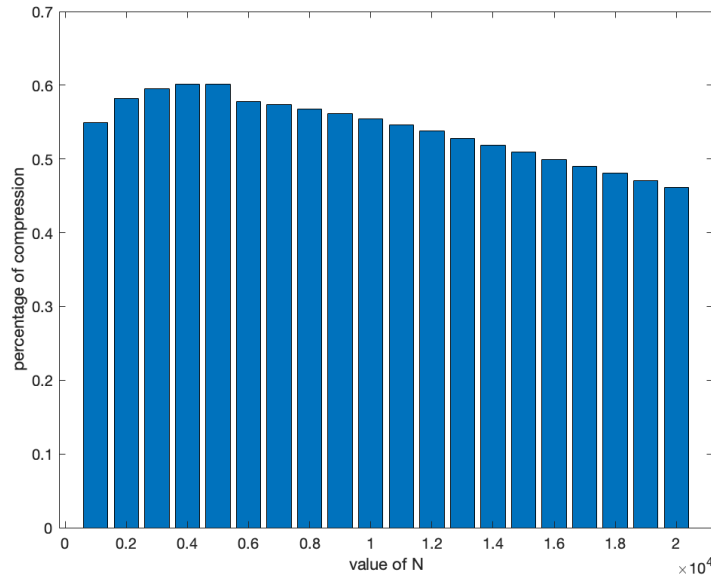


Figure 5: memory usage compression percentage based on N

```

1 % Read the content of the text file
2 fileID = fopen('warandpeace.txt', 'r');
3 textData = fscanf(fileID, '%c');
4 fclose(fileID);
5
6 %% Here we preprocess the data (upper casing, removing accents, ...)
7 words = preprocess(textData);
8
9 %% Building a dictionary with words and occurrences of each word
10 dict = configureDictionary("string","double");
11 for i = 1:length(words)
12     word = words{i};
13     if isKey(dict, word)
14         dict(word) = dict(word) + 1;
15     else
16         dict(word) = 1;
17     end
18 end
19
20 keys = dict.keys;
21 counts = dict.values;
22
23 %% Calculating entropy rate
24 entropy_rate = calculate_entropy_rate(counts);
25 disp('entropy rate = ');
26 disp(entropy_rate);
27
28 % sort the dictionary based on frequency
29 [sortedCounts, sortedIdx] = sort(counts, 'descend');
30 sortedKeys = keys(sortedIdx);
31
32 %% Encode the data with different values of N=(1000, 2000, ..., 20000)
33 Ns = 1000:1000:20000;
34 memory_percentage = zeros(length(Ns), 1);
35 for i = 1:length(Ns)

```

```

36     N = Ns(i);
37
38     dict_huffman = dictionary(sortedKeys(1:N), sortedCounts(1:N));
39
40     % Encoding the first N frequent words with huffman and writing the
41     % huffman tree codes in "HUFFMAN.TXT"
42     encode_huffman(dict_huffman);
43
44     % this function starts the process of encoding all the given text
45     encoded_data = start_encoding(words);
46
47     % save encoded binary data in "ENCODED_DATA.BIN"
48     save_encoded_data(encoded_data);
49
50     % calculating the percentage of memory usage decrease
51     memory_percentage(i) = 1 - (dir('encoded_data.bin').bytes+dir('
huffman.txt').bytes) / dir('warandpeace.txt').bytes;
52
53 end
54
55 % plot the comparison of memory usage for different values of N
56 bar(Ns, memory_percentage);
57 xlabel('value of N');
58 ylabel('percentage of compression')
59
60 % Functions
61
62 %% this function saves encoded data in binary format in 'encoded_data.
bin'
63 function save_encoded_data(encoded_data)
64     encoded_data = char(encoded_data);
65     encoded_data = [encoded_data, repmat('0', 1, 8-mod(strlength(
encoded_data), 8))];
66     binary_encoded_data = uint8(bin2dec(reshape(encoded_data, 8, []).'))
);
67     fileID = fopen('encoded_data.bin', 'w');
68     fwrite(fileID, binary_encoded_data, 'uint8');
69     fclose(fileID);
70 end
71
72 %% this function encodes all the given data. First it checks if any
word is
73 % Encoded by huffman and uses it's encoded value, then if it was not in
the
74 % Huffman tree, the word will be encoded with maxlength encoding.
75 function encoded_data = start_encoding(context)
76     encoded_data = '';
77     huffman_encoded_words = load_huffman_code();
78
79     for i = 2:length(context)
80         if isKey(huffman_encoded_words, context{i})
81             encoded_data = encoded_data + huffman_encoded_words(context
{i});
82         else
83             encoded_data = encoded_data + encode_maxlength(context{i});
84         end
85     end
86 end

```

```

87 end
88
89 %% this function loads the saved huffman tree from 'huffman.txt'
90 function huffman_encoded_words = load_huffman_code()
91     huffman_encoded_words = configureDictionary("string","string");
92     fileID = fopen('huffman.txt', 'r');
93     while ~feof(fileID)
94         line = fscanf(fileID,'%s,%s');
95         if strlength(line) == 0
96             fclose(fileID);
97             return;
98         end
99         line = split(line, ',');
100         huffman_encoded_words(line{1}) = line{2};
101     end
102     fclose(fileID);
103 end
104
105 %% this function encodes a given word by encoding each of it's
106     characters
107 % in 6 bits, and then add the length of encoded word in 8 bits, finally
108 % it adds a flag bit "1" to the encoded word that shows this is a ml
109 % encoding
110 function encoded_word = encode_maxlength(word)
111     alphabet = strcat(char(48:57), char(65:90));
112     encoded_word = '';
113
114     for j = 1:strlength(word)
115         % we use 6 bits because we have 36 characters
116         binary_character = dec2bin(find(alphabet == word(j)) - 1, 6);
117         encoded_word = strcat(encoded_word, binary_character);
118     end
119     encoded_word = "1" + dec2bin(length(encoded_word), 8) +
120     encoded_word;
121 end
122
123 %% this function encodes a list of words with huffman tree, in each
124     step
125 % it extracts the two words with minimum frequency and combines them
126 % and at the end each word will have a huffman encoding. then the
127     length of
128 % encoding will be added in 6 bits and finally a flag bit "0" also will
129 % show that this is a huffman encoding
130 function encode_huffman(words)
131     encoded_words = configureDictionary("string","string");
132
133     fileID = fopen('huffman.txt', 'w');
134
135     while length(words.values)~=1
136         [min1_value, min1_word] = findMin(words);
137         words = remove(words, min1_word);
138
139         for w = min1_word.split(",")
140             if isKey(encoded_words, w)
141                 encoded_words(w) = encoded_words(w) + "0";
142             else
143                 encoded_words(w) = "0";
144             end
145         end
146     end

```

```

141     end
142
143     [min2_value, min2_word] = findMin(words);
144     words = remove(words, min2_word);
145
146     for w = min2_word.split(",")
147         if isKey(encoded_words, w)
148             encoded_words(w) = encoded_words(w) + "1";
149         else
150             encoded_words(w) = "1";
151         end
152     end
153
154     words(min1_word+","+min2_word) = min1_value + min2_value;
155 end
156
157 keys = encoded_words.keys;
158 values = encoded_words.values;
159
160 max_digits_of_huffman = max(strlength(values));
161 length_bits_huffman = ceil(log2(max_digits_of_huffman));
162
163 for i = 1:length(keys)
164     encoded_words(keys{i}) = "0" + dec2bin(length(values{i}),
length_bits_huffman) + values{i};
165 end
166
167 keys = encoded_words.keys;
168 values = encoded_words.values;
169
170 keys = flip(keys);
171 values = flip(values);
172
173 for i = 1:length(encoded_words.keys)
174     fprintf(fileID, '%s,%s\n', keys(i), values(i));
175 end
176
177 fclose(fileID);
178 end
179
180 %% this function extracts the word with lowest frequency from
dictionary
181 % it is used in huffman algorithm
182 function [value, word] = findMin(array)
183     values = array.values;
184     words = array.keys;
185
186     value = values(1);
187     word = words(1);
188     for i = 1:length(values)
189         if values(i) < value
190             value = values(i);
191             word = words(i);
192         end
193     end
194 end
195
196 %% this function calculates the entropy rate given frequencies

```

```

197 function entropy_rate = calculate_entropy_rate(counts)
198     total_counts = sum(counts);
199     entropy_rate = 0;
200     for i = 1:length(counts)
201         probability = counts(i)/total_counts;
202         entropy_rate = entropy_rate - probability * log2(probability);
203     end
204 end
205
206 %% this function is used for preprocessing(uppercasing, removing
    accents
207 % , removing punctuations, split the text into words)
208 function words = preprocess(textData)
209     % Remove punctuation and convert text to uppercase
210     textData = upper(textData);
211     accents = {'Á', 'É', 'Í', 'Ó', 'Ú'};
212     non_accents = {'A', 'E', 'I', 'O', 'U'};
213
214     for i = 1:length(accents)
215         textData = regexprep(textData, accents{i}, non_accents{i});
216     end
217
218     textData = regexprep(textData, '[^0-9A-Z\s]', '');
219     textData = regexprep(textData, '\s+', ' ');
220
221     % Split the text into words
222     words = strsplit(textData);
223 end

```